2 академических часа · Для начинающих

Лекция №1. Введение в контейнеризацию

Что такое Docker и зачем он нужен

Чему научимся сегодня

- Поймём, почему программа может работать на одном ПК и не работать на другом.
- Узнаем, что такое контейнер и зачем он нужен.
- Сравним Docker с виртуальной машиной.
- Разберём понятия: образ, контейнер, Docker Engine и Docker Hub.
- Подготовимся к установке Docker на Debian 13.

Пять вопросов лекции

1	**Шаг 1**	1. Почему возникают проблемы при переносе программ?
2	**Шаг 2**	2. Что такое контейнеризация?
3	**Шаг 3**	3. Чем контейнер отличается от виртуальной машины?
4	**Шаг 4**	4. Как устроен Docker?
5	**Шаг 5**	5. Где Docker применяют на практике?

Наш учебный пример

- Мы написали небольшой сайт на Node.js.
- На компьютере разработчика сайт работает.
- Нужно перенести сайт на другой компьютер или сервер.
- На новом компьютере могут отличаться версия Node.js и настройки.
- Docker помогает переносить приложение вместе с нужным окружением.

Почему программы не всегда запускаются

Один проект — разные компьютеры

Знакомая ситуация

- Окружение — это программы, библиотеки и настройки, необходимые для работы приложения.
- У разработчика программа запускается без ошибок.
- На компьютере преподавателя программа выдаёт ошибку.
- Файлы проекта одинаковые, но окружение может отличаться.
- Например, не установлен нужный пакет или отличается версия Node.js.

Что нужно обычному Node.js-приложению?

- Исходный код приложения.
- Node.js подходящей версии.
- Установленные зависимости npm.
- Переменные окружения и другие настройки.
- Подключение к базе данных, если оно требуется.

Пример ошибки с версиями

- На первом компьютере используется современная версия Node.js.
- На другом установлена устаревшая версия.
- Код использует возможности, которых нет в старой версии.
- Результат — ошибка запуска или выполнения.

Обычный перенос программы

- Копируем исходный код на другой компьютер.
- Проверяем, установлен ли Node.js.
- Устанавливаем зависимости и меняем настройки.
- Если что-то забыли, получаем ошибку.
- Эти действия приходится повторять на каждом новом компьютере.

Как помогает Docker?

- Описываем или выбираем готовое окружение приложения.
- Создаём Docker-образ.
- Запускаем контейнер из образа на другом компьютере.
- Приложение использует окружение, подготовленное в образе.
- При этом настройки сети, внешних сервисов и самого компьютера всё ещё имеют значение.

Что такое контейнеризация?

Объяснение без сложных терминов

Контейнер — простыми словами

- Контейнер — изолированное окружение для запуска приложения.
- Контейнер — это способ запустить программу в изолированном окружении.
- В контейнере есть нужные файлы и зависимости приложения.
- Программа работает как отдельный процесс или набор процессов.
- При этом контейнер использует ядро операционной системы, на которой запускается.

Пример из жизни

- Представьте коробку с набором для сборки.
- Внутри находятся детали и инструкция.
- Вам не нужно искать каждую деталь по отдельности.
- Docker-образ похож на подготовленный комплект, а контейнер — на запущенный экземпляр.

Что Docker НЕ делает автоматически

- Не исправляет ошибки в исходном коде.
- Не заменяет резервное копирование важных данных.
- Не гарантирует работу без нужной сети и внешних сервисов.
- Не делает контейнер полностью независимым от операционной системы.

Docker-образ (один образ)

Шаблон, который содержит файловую систему, приложение, зависимости и настройки.



- Файловая система Linux
- Приложение (например, Nginx)
- Зависимости (библиотеки)
- Настройки по умолчанию

Создание контейнера из образа

Создание контейнера из образа

Создание контейнера из образа



Контейнер 1
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)



Контейнер 2
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)



Контейнер 3
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)

Один образ — несколько контейнеров

- Допустим, у нас есть один образ Nginx.
- Из него можно создать первый контейнер.
- Из того же образа можно создать второй контейнер.
- Каждый контейнер существует отдельно.
- Один контейнер можно остановить, не останавливая другой.

Что такое изоляция?

- Контейнеры могут видеть собственный список процессов.
- Каждый контейнер может иметь собственную файловую систему.
- Сетевое окружение контейнера можно отделить от сети хоста.
- Изоляция не равна отдельному компьютеру и не является абсолютной защитой.

Контейнер или виртуальная машина?

Похожая задача, но разное устройство

Что такое виртуальная машина?

- Виртуальная машина — программно созданный компьютер.
- Для неё выделяют виртуальные процессор, память и диск.
- Внутри устанавливают собственную операционную систему.
- Например: Windows 11 → VirtualBox → Debian 13.

Как устроена виртуальная машина?

- Есть физический компьютер (хост).
- Есть программа виртуализации, например VirtualBox.
- Внутри работает гостевая операционная система Debian.
- В Debian можно устанавливать обычные программы и службы.

Как устроен контейнер?

- Docker Engine работает в среде подходящей операционной системы.
- Контейнеры запускаются как изолированные процессы.
- Linux-контейнеры используют ядро Linux.
- Им обычно не нужна отдельная полноценная гостевая ОС для каждого приложения.

Архитектура ВМ и Docker-контейнеров

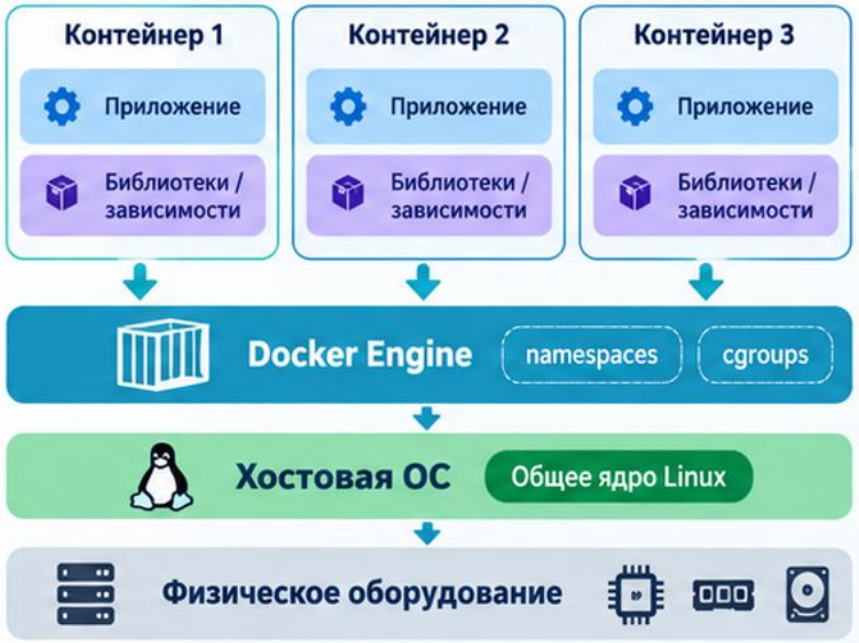
Сравнение уровней изоляции, компонентов и использования ресурсов

Виртуальные машины (VM)



- Каждая ВМ содержит собственную гостевую ОС
- Более высокая изоляция
- Больше потребление RAM и CPU
- Запуск обычно медленнее

Docker-контейнеры



- Контейнеры используют общее ядро хоста
- Лёгкая изоляция процессов
- Меньше потребление ресурсов
- Быстрый запуск

Критерий	Виртуальные машины (VM)	Docker-контейнеры
Изоляция	сильная	на уровне процессов
ОС внутри	полная гостевая ОС	нет отдельной гостевой ОС
Запуск	медленнее	быстрее
Ресурсы	больше	меньше
Типичное применение	разные ОС, полная виртуализация	развёртывание приложений

VM виртуализируют целую систему, а Docker контейнеризирует приложение и его зависимости.

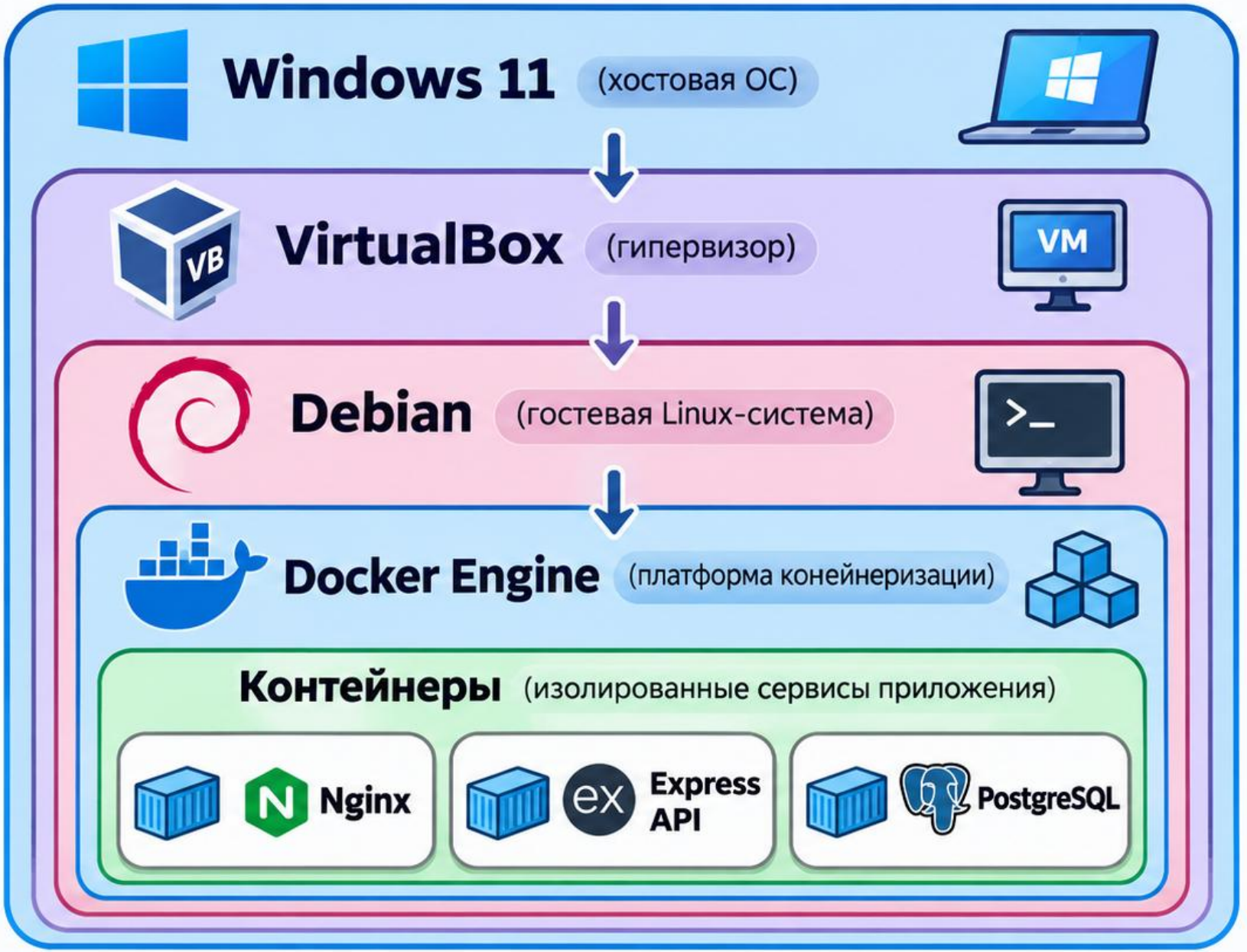
Виртуальная машина и контейнер

Признак	Виртуальная машина	Контейнер
Операционная система	Своя гостевая ОС	Общее ядро хоста
Запуск	Обычно медленнее	Обычно быстрее
Размер	Часто больше	Часто меньше
Изоляция	Граница виртуальной машины	Изоляция процессов
Типичная задача	Запустить отдельную ОС	Запустить приложение

Можно ли использовать Docker в VirtualBox?

- Да. На занятиях можно установить Debian 13 в VirtualBox.
- Затем установить Docker Engine внутри Debian.
- Контейнеры будут запускаться внутри этой Debian.
- Получается: Windows → VirtualBox → Debian → Docker → контейнер.

Наша учебная среда: Windows → VirtualBox → Debian → Docker



Windows 11 — хостовая ОС.
Основная операционная система на вашем компьютере.

VirtualBox — гипервизор.
Программа для создания и управления виртуальными машинами (ВМ).

Debian — гостевая Linux-система.
Виртуальная машина с дистрибутивом Debian, в которой мы работаем и запускаем наши сервисы.

Docker Engine — платформа **контейнеризации.**
Среда для создания, запуска и управления контейнерами.

Контейнеры — изолированные **сервисы приложения.**
Отдельные, независимые окружения с нашими сервисами, например: Nginx (веб-сервер), Express API (бэкенд) и PostgreSQL (база данных).

Как устроен Docker?

Пять основных понятий

Запомним пять слов

- Docker Client — принимает наши команды.
- Docker Daemon — выполняет работу с образами и контейнерами.
- Docker Image — подготовленный образ.
- Docker Container — контейнер, созданный из образа.
- Registry — хранилище образов.

Docker Client

- Это программа, которой мы управляем Docker.
- Чаще всего используем команду docker в терминале.
- Например, docker --version показывает версию клиентской программы.
- Клиент передаёт запрос службе Docker, если команда требует работы с контейнерами.

Команда 1. Проверяем наличие Docker

```bash
docker --version
```

Что важно

- Команда показывает установленную версию Docker CLI.
- Она НЕ устанавливает Docker и НЕ запускает контейнер.
- До установки Docker может появиться сообщение command not found.

Docker Daemon

- Это фоновая служба, которая управляет контейнерами и образами.
- В Linux обычно управляется через systemd.
- Клиент отправляет ей запрос, например «создай контейнер».
- Daemon выполняет необходимые действия.

Команда 2. Проверяем службу

```bash
sudo systemctl status docker
```

Что важно

- systemctl — инструмент управления системными службами Linux.
- status docker показывает, запущена ли служба Docker.
- В исправно работающей системе ищем Active: active (running).
- Нажмите q, чтобы выйти из просмотра состояния.

Docker Client → Docker Daemon → Container

Как команда пользователя превращается в работающий контейнер

Docker Client

CLI / команда docker




```
docker run nginx
docker ps
docker images
```

Принимает команды пользователя.

Docker Daemon

dockerd



- Получает запрос от клиента
- Управляет образами
- Создаёт и запускает контейнеры
- Работает через Docker API

Фоновая служба на хост-системе, которая управляет Docker.

Container

Запущенный контейнер



- Приложение
- Библиотеки
- Изолированная среда

Выполняет задачу, например Nginx.

1. Пользователь вводит команду



2. Client отправляет запрос



3. Daemon обрабатывает запрос



4. Создаётся контейнер



5. Приложение работает



Что важно помнить

- **Client** — это интерфейс команд.
- **Daemon** — это служба Docker.
- **Container** — это запущенный экземпляр образа.

Client передаёт команду, Daemon выполняет её, Container запускает приложение.

Docker Image — образ

- Образ содержит набор файлов для запуска приложения.
- Например, существуют готовые образы Nginx и Node.js.
- Образ сам по себе не является запущенной программой.
- Один образ можно использовать много раз.

Docker Container — контейнер

- Контейнер создаётся из образа.
- После запуска внутри него выполняется приложение.
- Контейнер может работать или быть остановлен.
- Контейнер можно удалить, при этом исходный образ обычно останется.

Docker Registry — хранилище образов

- Образы можно загружать из удалённого хранилища.
- Популярный публичный сервис — Docker Hub.
- Например, образ hello-world можно получить из Docker Hub.
- Подключение к интернету необходимо для первой загрузки образа, если он не сохранён локально.

От Docker Hub до запущенного контейнера

Как образ загружается, хранится локально и превращается в работающий контейнер

1 Docker Hub

официальный реестр образов



nginx:stable

2 docker pull nginx:stable

загрузка образа на локальный компьютер

```
$ docker pull
  nginx:stable
```



3 Локальный образ

образ хранится в системе Docker как шаблон



Слои образа

Image ID: 4f7a2c9e3b1d

Тег: nginx:stable

4 docker run -d --name web1 -p 8080:80 nginx:stable

создание и запуск контейнера из образа

```
$ docker run -d \
    --name web1 \
    -p 8080:80 \
    nginx:stable
```



5 Запущенный контейнер



Контейнер **web1** работает

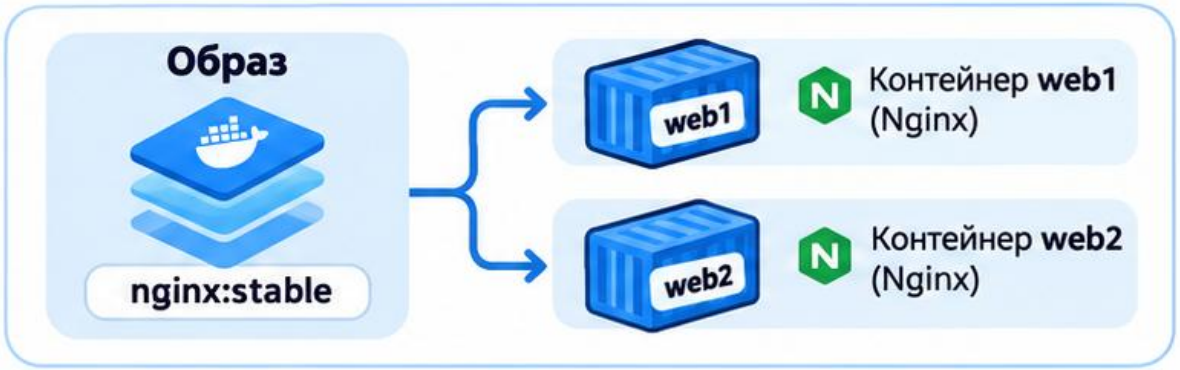
Внутри: Nginx

Опубликован порт: 8080 → 80



Что важно понять

- **Реестр** хранит образы.
- **Образ** — шаблон.
- **Контейнер** — экземпляр образа.
- **Один образ** можно использовать для создания нескольких контейнеров.



Что происходит при первом запуске контейнера?

- Мы вводим команду docker run.
- Docker проверяет, есть ли нужный образ на компьютере.
- Если образ отсутствует, Docker пытается его скачать.
- Docker создаёт и запускает контейнер.
- Программа внутри контейнера начинает выполняться.

Команда 3. Первый контейнер

```bash
sudo docker run hello-world
```

Что важно

- sudo — выполнение команды с правами администратора.
- docker run — создать и запустить контейнер.
- hello-world — название тестового образа.
- После успешного запуска появится Hello from Docker!; программа завершится.

Почему hello-world сразу остановился?

- hello-world запускается, печатает приветствие и завершает работу.
- У него нет задачи постоянно обслуживать запросы.
- Это нормальное поведение, а не ошибка.
- Программы вроде Nginx, наоборот, обычно работают долго.

Команда 4. Посмотреть контейнеры

BASH

```
sudo docker ps
sudo docker ps -a
```

Что важно

- docker ps — показывает контейнеры, которые сейчас работают.
- docker ps -a — показывает все контейнеры, в том числе завершившиеся.
- После hello-world контейнер обычно виден во втором списке.

Команда 5. Посмотреть образы

```bash
sudo docker images
```

Что важно

- Команда выводит сохранённые на компьютере образы.
- После первого запуска hello-world его образ обычно появится в списке.
- Образ и контейнер — не одно и то же.

Где применяют Docker?

От учебной практики до настоящих проектов

Docker при разработке

- Разработчик может запускать приложение в повторяемом окружении.
- Команда проекта может договориться об одинаковых версиях зависимостей.
- Удобнее проверять приложение на разных компьютерах.
- Но файлы проекта и конфигурацию всё равно нужно поддерживать.

Docker при тестировании

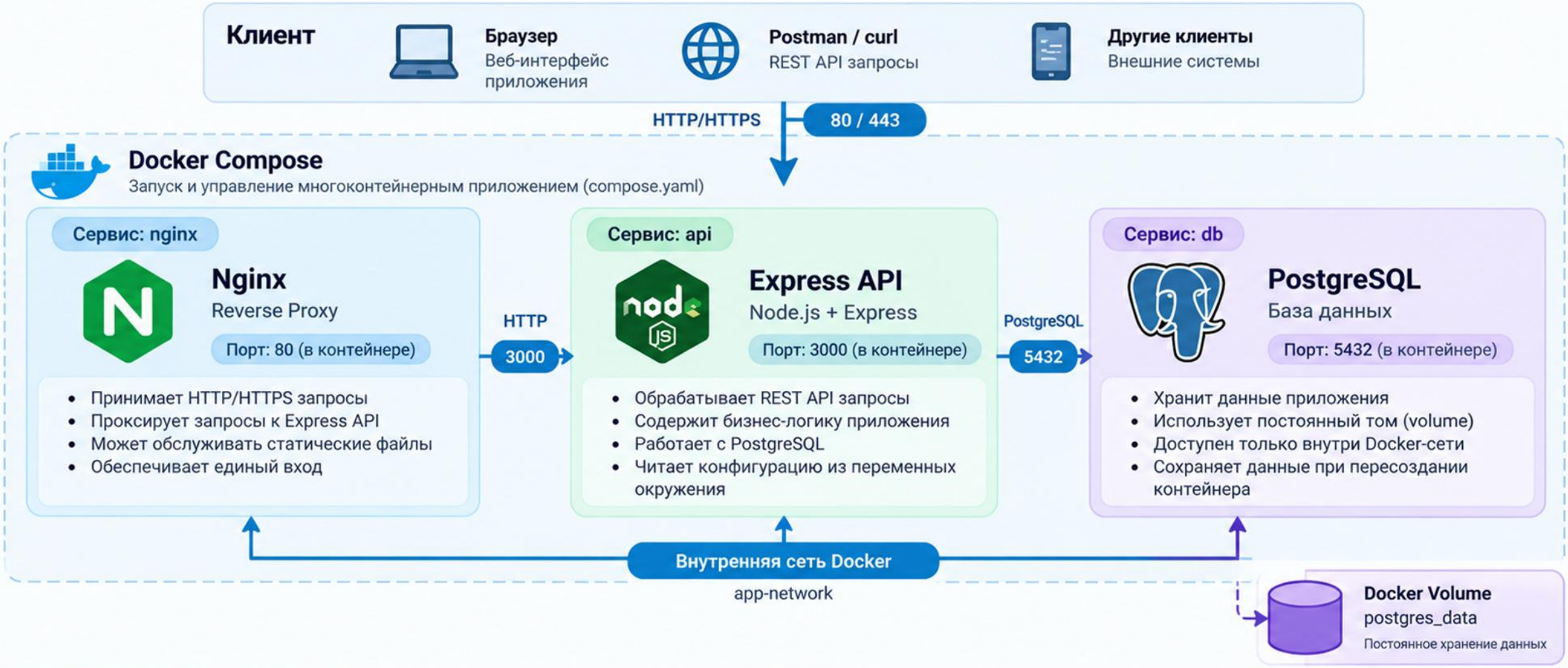
- Для проверки можно создать временный контейнер.
- В нём запускают тесты или вспомогательную службу.
- После проверки ненужный контейнер удаляют.
- Так легче повторить проверку в похожих условиях.

Docker на сервере

- В контейнере можно запустить веб-сервер Nginx.
- В другом контейнере — API на Express.
- В третьем — PostgreSQL.
- Позже мы научимся связывать такие сервисы.
- Для важных данных нужно отдельно продумать постоянное хранение.

Итоговый проект: веб-приложение на Docker

Архитектура и взаимодействие компонентов: Nginx → Express API → PostgreSQL



Ключевые преимущества итогового проекта

Единая команда запуска
docker compose up

Изолированные сервисы
Каждый компонент в отдельном контейнере

Сохраняемые данные
PostgreSQL в Docker Volume

Готово к развёртыванию
Можно запустить на Linux-сервере

Наш путь на следующих занятиях

- ЛР №1: установить Docker Engine и проверить hello-world.
- Лекция №2: подробно изучить команды запуска и управления.
- ЛР №2: запустить Nginx, посмотреть логи и управлять контейнерами.
- Далее: собственные образы, сети, тома и Docker Compose.

Что запомнить

1. Docker помогает запускать приложения в заранее подготовленном окружении.
2. Контейнер использует ядро хостовой ОС; это не отдельная виртуальная машина.
3. Image — образ; Container — созданный из него экземпляр.
4. Docker Client принимает команды, Docker Daemon управляет контейнерами.
5. В лабораторной №1 установим Docker на Debian 13 и проверим hello-world.

Вопросы для проверки

1. Почему приложение может работать на одном компьютере и не работать на другом?
2. Что такое контейнер простыми словами?
3. Чем контейнер отличается от виртуальной машины?
4. Что такое Docker Image и Docker Container?
5. Зачем нужен Docker Daemon?
6. Для чего используется Docker Hub?
7. Что делает команда docker --version?
8. Почему hello-world не остаётся работающим контейнером?
9. Что покажет docker ps -a?
10. Что мы будем делать в лабораторной №1?

От исходного кода к контейнеру

Как приложение и его зависимости превращаются в Docker-образ и запускаются в контейнере

1 Исходный код приложения

Разработчик пишет код и описывает зависимости.

```
my-app/
  src/
    server.js
    package.json
  .dockerignore
  Dockerfile
```

package.json (фрагмент)

```
{
  "name": "my-app",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

2 Dockerfile

Описывает, как собрать образ: базовый образ, копирование файлов, установка зависимостей и команда запуска.

Dockerfile

```
FROM node:22-alpine

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000
CMD ["node", "src/server.js"]
```

3 Сборка образа

Docker читает Dockerfile, выполняет инструкции и создаёт слоёный образ.

Команда сборки

```
$ docker build -t my-app:1.0 .
```

Процесс сборки

- Загрузка базового образа
- Установка зависимостей
- Копирование исходного кода
- Создание слоёв образа
- Готовый образ my-app:1.0

4 Docker-образ

Результат сборки — неизменяемый образ, состоящий из слоёв.



my-app:1.0
Docker Image

5 Запуск контейнера

Из образа создаётся контейнер, в котором запускается приложение.



Команда запуска

```
$ docker run -d \
    --name my-app \
    -p 3000:3000 \
    my-app:1.0
```

Контейнер запущен
Приложение доступно на http://localhost:3000



Исходный код
Файлы приложения и зависимости

→



Dockerfile
Инструкции для сборки

→



Docker-образ
Неизменяемый шаблон

→



Контейнер
Запущенное приложение в изолированной среде

Ядро Linux, namespaces и cgroups

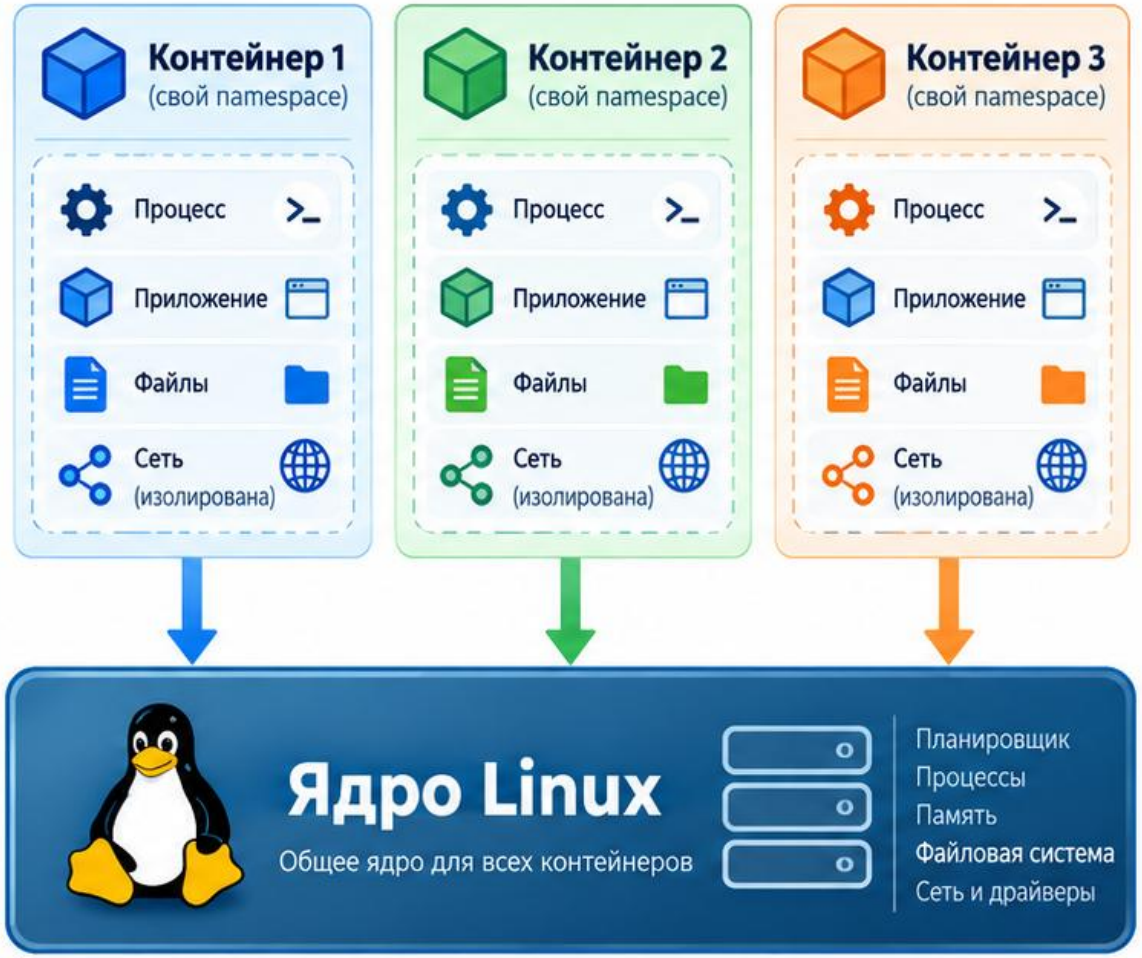
Как Linux-контейнеры обеспечивают изоляцию и контроль ресурсов

Namespaces = изоляция

Предоставляют контейнеру своё собственное представление о системных ресурсах.

- **PID** — изоляция процессов
- **NET** — изоляция сети
- **MNT** — изоляция точек монтирования
- **UTS** — имя хоста (hostname)
- **IPC** — межпроцессное взаимодействие
- **USER** — пользователи и группы

Каждый контейнер видит своё пространство ресурсов.



cgroups = контроль ресурсов

Позволяют ограничивать, учитывать и распределять использование ресурсов между контейнерами.

- **CPU** — ограничение процессора (50%)
- **RAM** — ограничение памяти (2 GB)
- **I/O** — контроль дисковых операций (100 MB/s)
- **PIDs** — лимит процессов (256)

Позволяют учитывать и ограничивать использование ресурсов.

1. **Контейнер =** изолированные процессы
2. **Namespaces** разделяют окружение
3. **cgroups** ограничивают ресурсы

Установка Docker в Debian

Пошаговая схема установки Docker Engine из официального APT-репозитория

1 Подготовка системы



- Обновить список пакетов
- Установить ca-certificates, curl, gnupg

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
```

2 Добавить ключ Docker



- Создать каталог /etc/apt/keyrings
- Скачать GPG-ключ

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3 Подключить репозиторий



- Использовать официальный репозиторий Docker
- Архитектура и версия Debian определяются автоматически

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian $(. /etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

4 Установить Docker Engine



- Установить Docker Engine, CLI и плагины
- Compose устанавливается как plugin

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

5 Проверить службу



- Проверить статус systemd
- Убедиться, что Docker запущен

```
sudo systemctl status docker
sudo docker version
sudo docker info
```

Служба: `dockerd`

6 Тестовый запуск



- Запустить hello-world
- Проверить, что образ скачан и контейнер выполнился

```
sudo docker run hello-world
```

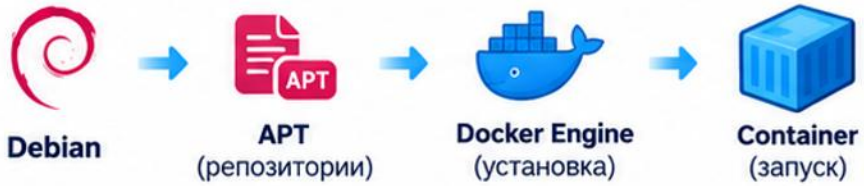
В выводе должно появиться:

`Hello from Docker!`

Что важно помнить

1. Используйте официальный репозиторий Docker.
2. На занятиях команды выполняем через `sudo`.
3. Группа `docker` даёт почти root-доступ.
4. Для проверки удобно использовать `hello-world` и `nginx`.

Краткая схема



Архитектура службы Docker

Как Docker CLI, Docker Daemon и контейнерные компоненты взаимодействуют в Linux

1. Слой пользователя

Пользователь



Примеры команд:

```
docker ps
docker run nginx
docker images
```

2. Слой клиента (CLI)

Docker CLI

Клиент командной строки

Unix socket / Docker API

3. Слой службы (Docker Engine)

Docker Daemon (dockerd)

Системная служба

- Принимает команды от клиента
- Управляет образами, контейнерами, сетями и томами
- Работает через Docker API
- Запускается как служба systemd

pull / push

Загрузка и публикация Docker-образов

Docker Registry / Docker Hub

Хранилище Docker-образов (публичное)

Локальные объекты Docker

- **Images** — Docker-образы
- **Networks** — Сети
- **Volumes** — Тома данных
- **Containers** — Контейнеры

4. Слой выполнения (runtime)

containerd

Управление жизненным циклом контейнеров

runc

Низкоуровневый запуск контейнера

5. Слой контейнеров

Контейнеры (изолированные процессы)

- nginx
- api
- db

Как всё работает? (последовательность шагов)

1. **Пользователь вводит команду** — Например, docker run nginx
2. **CLI отправляет запрос** — Через Unix socket (Docker API)
3. **dockerd обрабатывает запрос** — Управляет образами, контейнерами, сетями и томами
4. **containerd и runc запускают контейнер** — Создают и запускают контейнер
5. **Контейнер выполняет приложение** — Например, nginx, api или db в изолированном окружении

Ключевые выводы:

- **CLI — интерфейс команд** — Позволяет пользователю управлять Docker.
- **dockerd — основная служба Docker** — Обрабатывает команды и управляет всеми объектами.
- **containerd и runc — внутренние компоненты запуска** — Отвечают за создание и запуск контейнеров.
- **Контейнеры — изолированные процессы** — Запускают приложения в собственном окружении, на общей Linux-системе.

Docker Compose

Контейнер 1 — Nginx

Отдаёт HTML, CSS и JS; проксирует /api

↓ Внутренняя сеть · python-api:8000

Контейнер 2 — Python

Встроенный http.server · GET и POST · JSON

↓

Docker Volume

/data/tasks.json — постоянное хранение задач